

ResiNet: Multi-Threaded Smart Microgrid P2P Energy Trading & Load Balancing System

Course Code:	CSE2006 - Programming in Java
Course Type:	LP (Lecture + Practical - Flipped Course)
Submission Mode:	GitHub Repository & Portal PDF Report
Domain & Topic:	Smart Grid / P2P Energy Trading / Java Concurrency
Date of Submission:	September 2026

Submitted for Continuous Academic Assessment & Evaluation

1. Introduction

The modern energy transition demands decentralized, intelligent, and real-time management of localized renewable energy resources. Traditional centralized utility grids suffer from transmission loss, line congestion, and inefficient pricing mechanisms when integrating distributed energy resources (DERs) such as residential solar panels, wind turbines, and battery energy storage systems (BESS).

ResiNet is a multi-threaded smart microgrid P2P energy trading and load balancing platform implemented in Java for the CSE2006 course. It empowers microgrid prosumers to directly trade excess energy with neighboring commercial and residential consumers in real-time. By utilizing advanced Java concurrent programming, custom priority queue matching algorithms, embedded SQLite JDBC persistence, and character/byte stream file logging, ResiNet delivers a robust computational framework for decentralized energy distribution.

2. Problem Statement & Objectives

Problem Statement:

Small-scale renewable energy producers (prosumers) currently face significant barriers monetizing surplus energy production due to rigid utility feed-in tariffs. Simultaneously, consumers pay high peak-hour retail rates. Existing software solutions lack real-time peer-to-peer (P2P) market clearing, multi-threaded grid overload safety guarantees, and transparent transactional audit trails.

Key Project Objectives:

- Develop a polymorphic Java object-oriented domain model for microgrid entities (Prosumer, Consumer, BESS).
- Implement a concurrent PriorityQueue-based order matching engine that matches buy and sell orders based on dynamic price-time priority.
- Engineer a multithreaded simulation engine utilizing ReentrantLock synchronization to enforce grid transformer safety limits.
- Build an embedded SQLite database persistence layer using JDBC PreparedStatements for financial transaction records.
- Provide Java File I/O stream utilities to export telemetry audit logs and text reports.

3. Functional Requirements

Module Name	Functional Description	Target Output / Result
1. Node Management	Register, update, and monitor microgrid nodes (Solar, Wind, Battery, Registry, Status, Telemetry)	Nodes, Bidder, Registry, Status, Telemetry
2. P2P Order Matcher	Concurrent matching of buy bids and sell asks via PriorityQueue-based matching engine	Matched Energy Transactions & Price Execution
3. Grid Simulation	Multithreaded generation fluctuation, demand modeling, Real-time Power Balance & Safety Alarms	Grid Load, Power Balance, Safety Alarms
4. DB & Analytics	JDBC persistence for financial transaction logs, SQL as Query Tables (energy_nodes, energy_transa	SQLite Tables (energy_nodes, energy_transa

5. Report Exporter	File I/O character stream export of audit telemetry logs to <code>resinnet_audit_report.txt</code> & Log Files
--------------------	--

4. Non-Functional Requirements

- 1. Performance & Concurrency:** Sub-millisecond order matching latency using Java `PriorityQueue` and thread-safe `ConcurrentHashMap` registries under concurrent load.
- 2. Data Integrity & Reliability:** SQLite database transactions with rollback capability on error and atomic operations.
- 3. Security & Validation:** Strict validation of order parameters (non-negative power/price) and custom exception handling.
- 4. Maintainability & Modularity:** Clear package layout (`com.smartgrid.resinet`), SOLID design principles, and comprehensive docstrings.

5. System Architecture & Diagram

ResiNet adopts a layered modular architecture consisting of the Presentation Layer (Interactive CLI), Core Engine Layer (Multithreaded Simulation & Order Matcher), Data Access Layer (JDBC DAOs), and Storage Layer (SQLite DB & Log Files).

```
+-----+ | ResiNet
Main CLI Dashboard |
+-----+ |
+-----+ | |
+-----v-----+ +-----v-----+ |
GridSimulationEngine Thread | | OrderMatchingEngine | | (ReentrantLock & Solar Fluctuation) | |
(PriorityQueue Buy/Sell Queues) | +-----+
+-----+ +-----+ | +-----v-----+
+-----v-----+ +-----+ | Polymorphic Node Domain | | JDBC Database Access Layer |
| (Prosumer, Consumer, BatteryStorage) | | (EnergyNodeDAO & TransactionDAO) |
+-----+ +-----+ +-----+ | |
+-----v-----+ +-----v-----+ |
TelemetryLoggerThread (I/O) | | SQLite Embedded Database | | (logs/grid_telemetry.log) | |
(data/resinet_grid.db) | +-----+
+-----+
```

6. System Design Diagrams

6.1 Use Case Diagram Description:

- **Prosumer Actor:** Submits sell orders, streams solar/wind power generation telemetry.
- **Consumer Actor:** Submits buy bids, monitors power consumption and demand fulfillment.
- **Grid Operator Actor:** Toggles multithreaded simulation, monitors transformer capacity, views JDBC logs.

6.2 Class Diagram & Entity Hierarchy:

- Abstract Base Class EnergyNode (nodeId, name, locationZone, currentPowerKW).
- Subclasses ProsumerNode, ConsumerNode, BatteryStorageNode.
- Interfaces Tradeable (deductPower, addPower) and MonitoredDevice (getHealthIndex).
- Service Layer GridSimulationEngine, OrderMatchingEngine, DatabaseManager.

6.3 Database ER Diagram Schema:

- Table energy_nodes (node_id PK, name, node_type, location_zone, current_power_kw, is_active).
- Table energy_transactions (transaction_id PK, seller_node_id FK, buyer_node_id FK, matched_power_kw, price_per_kwh, total_cost, timestamp).

7. Design Decisions & Rationale

1. Embedded SQLite Database over Standalone MySQL/PostgreSQL: SQLite provides zero-configuration embedded persistence inside a single file (`data/resinet\_grid.db`), eliminating the need for evaluators to install

external database services while fully demonstrating JDBC `PreparedStatement` and transaction concepts.

2. PriorityQueue for Order Matching: Java's `PriorityQueue` with custom `Comparable` implementation naturally sorts buy bids descending (highest bid first) and sell asks ascending (lowest price first), enabling $O(\log N)$ order insertion and $O(1)$ top-of-book matching.

3. ReentrantLock for Grid Load Concurrency: `ReentrantLock` was chosen over standard synchronized blocks for grid load checks because it allows explicit lock polling, fairness control, and clean try-finally release semantics under high multithreaded simulation loads.

8. Implementation Details & Source Code Highlights

8.1 Core Package Structure:

ResiNet consists of 14 modular `.java` files structured into clean packages under `com.smartgrid.resinet`:

- **com.smartgrid.resinet.model**: `EnergyNode`, `ProsumerNode`, `ConsumerNode`, `BatteryStorageNode`, `TradeOrder`, `EnergyTransaction`
- **com.smartgrid.resinet.interfaces**: `Tradeable`, `MonitoredDevice`
- **com.smartgrid.resinet.exceptions**: `ResiNetException`, `GridOverloadException`, `InsufficientEnergyException`, `InvalidTradeOrderException`
- **com.smartgrid.resinet.engine**: `GridSimulationEngine`, `OrderMatchingEngine`, `TelemetryLoggerThread`
- **com.smartgrid.resinet.db**: `DatabaseManager`, `EnergyNodeDAO`, `TransactionDAO`
- **com.smartgrid.resinet.util & test**: `ReportGenerator`, `ResiNetTestSuite`, `Main`

8.2 PriorityQueue Order Matching Snippet:

```
// PriorityQueue Order Matching Logic (OrderMatchingEngine.java)
if (topBuy.getPricePerKWh() >= topSell.getPricePerKWh()) {
    double tradeVolumeKW = Math.min(topBuy.getPowerAmountKW(),
    topSell.getPowerAmountKW());
    double executionPrice = (topBuy.getPricePerKWh() +
    topSell.getPricePerKWh()) / 2.0;
    EnergyTransaction tx = new EnergyTransaction(
    "TX-" + UUID.randomUUID().toString().substring(0, 8),
    topSell.getNodeId(), topBuy.getNodeId(),
    tradeVolumeKW, executionPrice );
    transactionDAO.insertTransaction(tx);
    // JDBC Persistence
}
```

9. Verification & Execution Results

The system was compiled and executed using JDK 26 and SQLite JDBC 3.45. Below is the verified execution trace:

```
=== RESINET CONTROL CENTER MENU ===
1. View Microgrid Telemetry Dashboard
2. Submit Custom P2P Energy Trade Order
3. Toggle Multithreaded Live Grid Simulation
4. View Database Transaction Audit Log (JDBC)
5. Register New Microgrid Energy Node
6. Export Telemetry & Audit Report (File I/O)
7. Run Automated System Unit Test Suite
8. Exit Platform
--- LIVE MICROGRID TELEMTRY DASHBOARD ---
NODE ID | NAME | TYPE | POWER/DEMAND | STATUS
-----
NOD-SOL-01 | Greenfield Solar Park | PROSUMER | 250.00 kW | ACTIVE
NOD-WND-02 | Coastal Wind Farm | PROSUMER | 400.00 kW | ACTIVE
NOD-BES-03 | Central Storage BESS | BATTERY_STOR | SOC: 75.0% | ACTIVE
NOD-RES-04 | Sunset Valley Residential | CONSUMER | Req: 180.0kW | ACTIVE
NOD-IND-05 | Apex Industrial Hub | CONSUMER | Req: 350.0kW | ACTIVE
Total Generation: 650.00 kW | Total Demand: 530.00 kW
```

10. Testing Approach & Automated Test Suite

An automated test suite (`ResiNetTestSuite`) was executed to validate system correctness across 4 key categories:

Test Category	Test Description	Assertion / Expected Result	Status
OOP & Polymorphism	Polymorphic type identification & getProsumers()	Prosumers & Consumer types match contract	PASS

Exception Handling	Verify power deficit throws InsufficientEnergyException	Correctly throws ERR_INSUFFICIENT_ENERGY	PASS
Order Matching	PriorityQueue pairing of buy bids & sell asks	Successful transaction execution & price calc	PASS
Order Validation	Negative power order validation check	InvalidTradeOrderException thrown correctly	PASS
JDBC & Database	SQLite PreparedStatement insertion & query	Active node count > 0 from database query	PASS

11. Challenges Faced & Technical Solutions

- 1. Concurrency Control in PriorityQueue Order Matching:** Standard `PriorityQueue` is non-thread-safe. Under multithreaded simulation, concurrent additions produced race conditions. *Solution:* Wrapped queue operations inside synchronized methods and used `ReentrantLock` for state updates.
- 2. SQLite JDBC Driver Dependency Resolution:** Running on evaluator machines without Maven requires self-contained dependencies. *Solution:* Embedded `sqlite-jdbc.jar` and `slf4j` helper JARs inside `lib/` and wrote single-click `.bat` and `.sh` build scripts specifying local classpath arguments.
- 3. Dynamic Load Balancing Alarms:** Detecting transformer capacity overloads during simulation required custom exception handling without breaking the background loop. *Solution:* Threw `GridOverloadException` caught by the logger thread while maintaining thread safety.

12. Academic Learnings & Key Takeaways

- Enhanced expertise in Java Object-Oriented paradigm (Abstract Classes, Interfaces, Method Overriding, Encapsulation).
- Mastered Java Multithreading mechanics (Thread lifecycle, Runnable interface, synchronized synchronization locks, ReentrantLock).
- Implemented production-grade Database Applications using JDBC PreparedStatement, Connection Pooling, and SQLite DDL/DML.
- Mastered Java I/O Streams (Character streams BufferedWriter/PrintWriter & Byte streams) for telemetry audit logs.

13. Future Enhancements & Roadmap

- **Machine Learning Integration:** Add predictive neural network models (e.g. LSTM) to forecast solar generation 24 hours ahead.
- **Spring Boot REST API & Web Dashboard:** Expose microgrid telemetry via REST APIs and build a React / Tailwind CSS interactive visual dashboard.
- **Smart Contract / Blockchain Ledger:** Replace central SQLite DB with Hyperledger Fabric smart contracts for tamper-proof energy trading settlement.

14. References & Bibliography

1. Herbert Schildt, *Java: The Complete Reference*, 11th Edition, Oracle Press, 2018.
2. Cay S. Horstmann, *Core Java Vol I—Fundamentals*, 11th Edition, Pearson, 2019.
3. SQLite JDBC Driver Documentation & Repository: <https://github.com/xerial/sqlite-jdbc>
4. Oracle Java SE 17 / 21 Concurrency Utilities Specification (`java.util.concurrent`).